



PRACTICAL LABORATORY MANUAL

Database Management System

Course Code : _____

Program : S.Y.B.Sc. AIML

Semester : III

Academic Year : 2026–2027

Prepared By

Poonam Parmar

List of Practicals

The following practical exercises are included in this laboratory manual.

Practical No.	Practical Title	Page No.
01	Create and manage databases and tables: • Use DDL commands to create tables for Department, Employee, Product, and Sales • Insert sample data into each table	
02	Apply constraints: • Add primary keys and foreign keys • Apply NOT NULL, UNIQUE, CHECK constraints	
03	Perform basic SQL operations: • SELECT queries • WHERE clause • ORDER BY, GROUP BY, HAVING	
04	Use aggregate functions: • COUNT, SUM, AVG, MIN, MAX • Combine with GROUP BY	
05	Perform JOIN operations: • INNER JOIN • LEFT JOIN • RIGHT JOIN	
06	Work with subqueries and nested queries	
07	Design an ER diagram for a sample application and map it to relational tables	
08	Normalize a set of unnormalized relations to 1NF, 2NF, 3NF, and BCNF	
09	Managing Transactions in SQL using COMMIT, ROLLBACK, and SAVEPOINT	

Practical titles and page numbers will be updated during the final compilation of the laboratory manual.

Artificial Intelligence and Machine Learning | Semester III

Practical 1: Creating and Managing Databases and Tables

Title

Creating a database and defining tables for Department, Employee, Product, and Sales using DDL commands, and inserting sample data using DML commands.

Aim

To understand and apply Data Definition Language (DDL) commands to create a database and design relational tables, and to use Data Manipulation Language (DML) commands to populate these tables with sample data.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 (GUI client) or MySQL Command Line Client

Theory

What is a Database and a DBMS?

A database is an organized collection of related data stored electronically so that it can be easily accessed, managed, and updated. A Database Management System (DBMS) is the software that sits between the user and the database, allowing users to create, read, update, and delete data in a controlled and efficient manner. A Relational DBMS (RDBMS) — such as MySQL, Oracle, or PostgreSQL — organizes data into tables (also called relations), where each table consists of rows (records/tuples) and columns (fields/attributes).

Categories of SQL Commands

Structured Query Language (SQL) commands used to interact with a relational database are grouped into the following categories:

- DDL (Data Definition Language): Defines and modifies the structure of database objects. Examples: CREATE, ALTER, DROP, TRUNCATE.
- DML (Data Manipulation Language): Used to manipulate the data stored in tables. Examples: INSERT, UPDATE, DELETE.

Artificial Intelligence and Machine Learning | Semester III

- DQL (Data Query Language): Used to retrieve data. Example: SELECT.
- DCL (Data Control Language): Controls access/permissions. Examples: GRANT, REVOKE.
- TCL (Transaction Control Language): Manages transactions. Examples: COMMIT, ROLLBACK, SAVEPOINT.

CREATE DATABASE and USE Statements

Before creating any table, a database (a named container for a group of related tables) must be created using `CREATE DATABASE database_name;` and selected as the active database using `USE database_name;`. All subsequent statements act on the currently selected database.

CREATE TABLE Statement

The `CREATE TABLE` statement defines a new table by specifying its name, along with a list of column names, each with a data type and, optionally, constraints. The general syntax is:

```
CREATE TABLE table_name (
    column1 datatype [constraint],
    column2 datatype [constraint],
    ...
);
```

Commonly Used MySQL Data Types

Data Type	Description	Example
INT	Whole (integer) numbers	DeptID INT
VARCHAR(n)	Variable-length string, up to n characters	EmpName VARCHAR(50)

Artificial Intelligence and Machine Learning | Semester III

Data Type	Description	Example
DECIMAL(p,s)	Fixed-point number with p total digits, s after decimal	UnitPrice DECIMAL(10,2)
DATE	Calendar date (YYYY-MM-DD)	DOJ DATE
CHAR(n)	Fixed-length string of exactly n characters	DeptCode CHAR(4)

INSERT INTO Statement

The INSERT INTO statement adds new rows of data into a table. Values must be listed in the same order as the columns (or column names can be specified explicitly, which is the safer practice):

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...);
```

The Database Used in This Manual

Across the Practicals in this manual, a single sample database named CompanyDB is used, consisting of four related tables: Department, Employee, Product, and Sales. This mirrors a simplified company scenario in which employees belonging to departments sell products, and each sale is recorded as a transaction.

Procedure

1. Launch MySQL Workbench (or the MySQL command-line client) and connect to the local MySQL server using the root/administrator credentials.
2. Open a new SQL query tab (query editor window).
3. Create a new database named CompanyDB using the CREATE DATABASE statement.
4. Select CompanyDB as the active database using the USE statement.
5. Write and execute a CREATE TABLE statement for the Department table.
6. Write and execute a CREATE TABLE statement for the Employee table.

Artificial Intelligence and Machine Learning | Semester III

7. Write and execute a CREATE TABLE statement for the Product table.
8. Write and execute a CREATE TABLE statement for the Sales table.
9. Insert at least five sample rows into each of the four tables using INSERT INTO statements.
10. Verify the table structures using the DESCRIBE (or DESC) command.
11. Verify the inserted data using SELECT * FROM table_name; for each table.

Program / Commands

Step 1–2: Create and select the database

```
-- Create the database
CREATE DATABASE CompanyDB;

-- Select the database for subsequent statements
USE CompanyDB;
```

Step 3: Create the Department table

```
CREATE TABLE Department (
    DeptID INT,
    DeptName VARCHAR(50),
    Location VARCHAR(50)
);
```

Step 4: Create the Employee table

```
CREATE TABLE Employee (
    EmpID INT,
    EmpName VARCHAR(50),
    DeptID INT,
    Designation VARCHAR(30),
    Salary DECIMAL(10,2),
    DOJ DATE
);
```

Step 5: Create the Product table

```
CREATE TABLE Product (
    ProductID INT,
    ProductName VARCHAR(50),
    Category VARCHAR(30),
    UnitPrice DECIMAL(10,2)
);
```

Artificial Intelligence and Machine Learning | Semester III

Step 6: Create the Sales table

```
CREATE TABLE Sales (  
    SaleID    INT,  
    ProductID INT,  
    EmpID     INT,  
    Quantity  INT,  
    SaleDate  DATE,  
    TotalAmount DECIMAL(10,2)  
);
```

Step 7: Insert sample data

```
-- Department records  
INSERT INTO Department (DeptID, DeptName, Location) VALUES  
(1, 'Sales',    'Mumbai'),  
(2, 'IT',      'Pune'),  
(3, 'HR',      'Delhi'),  
(4, 'Finance', 'Bengaluru'),  
(5, 'Marketing', 'Chennai');  
  
-- Employee records  
INSERT INTO Employee (EmpID, EmpName, DeptID, Designation, Salary, DOJ) VALUES  
(101, 'Aditi Sharma', 1, 'Sales Executive', 45000.00, '2021-06-14'),  
(102, 'Rohan Mehta', 2, 'Software Engineer', 65000.00, '2020-03-01'),  
(103, 'Kavya Iyer', 3, 'HR Manager', 58000.00, '2019-11-20'),  
(104, 'Sameer Khan', 4, 'Accountant', 50000.00, '2022-01-10'),  
(105, 'Neha Verma', 1, 'Sales Executive', 47000.00, '2021-08-05'),  
(106, 'Arjun Nair', 2, 'Software Engineer', 70000.00, '2018-07-23');  
  
-- Product records  
INSERT INTO Product (ProductID, ProductName, Category, UnitPrice) VALUES  
(201, 'Laptop',    'Electronics', 55000.00),  
(202, 'Office Chair', 'Furniture', 4500.00),  
(203, 'Printer',   'Electronics', 12000.00),  
(204, 'Desk',      'Furniture', 7000.00),  
(205, 'Monitor',   'Electronics', 9000.00);  
  
-- Sales records  
INSERT INTO Sales (SaleID, ProductID, EmpID, Quantity, SaleDate, TotalAmount) VALUES  
(301, 201, 101, 2, '2023-01-15', 110000.00),  
(302, 203, 105, 1, '2023-02-10', 12000.00),  
(303, 202, 101, 3, '2023-02-18', 13500.00),  
(304, 205, 105, 4, '2023-03-05', 36000.00),  
(305, 204, 101, 1, '2023-03-20', 7000.00);
```

Artificial Intelligence and Machine Learning | Semester III

Step 8–9: Verify structure and data

```
-- View table structure
DESCRIBE Department;
DESCRIBE Employee;
DESCRIBE Product;
DESCRIBE Sales;

-- View inserted data
SELECT * FROM Department;
SELECT * FROM Employee;
SELECT * FROM Product;
SELECT * FROM Sales;
```

Implementation

[Insert Figure 1 here — successful execution of CREATE DATABASE and CREATE TABLE statements]

[Insert Figure 2 here — DESCRIBE output for all four tables showing column structure]

*[Insert Figure 3 here — SELECT * output for all four tables showing inserted sample data]*

Conclusion

In this Practical, a new database, CompanyDB, was successfully created, and four relational tables — Department, Employee, Product, and Sales — were defined using DDL commands. Sample data was inserted into each table using DML commands, and the table structures and stored data were verified using DESCRIBE and SELECT statements.

Practical 2: Applying Constraints

Title

Applying primary key, foreign key, NOT NULL, UNIQUE, and CHECK constraints to the CompanyDB tables.

Aim

To understand the concept of integrity constraints in a relational database and to apply PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints while creating tables, thereby enforcing data accuracy and referential integrity.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

What Are Constraints?

Constraints are rules enforced on data columns of a table, used to limit the type of data that can go into a table. They ensure the accuracy, consistency, and reliability of the data stored in the database. Constraints can be applied at the column level (immediately after the data type) or at the table level (as a separate line, useful for constraints spanning multiple columns or given custom names).

Types of Constraints

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each row in a table. Combines NOT NULL + UNIQUE. A table can have only one primary key, which may consist of a single column or multiple columns (composite key).

Artificial Intelligence and Machine Learning | Semester III

Constraint	Purpose
FOREIGN KEY	Establishes a link between the data in two tables by referencing the primary key of another (or the same) table, enforcing referential integrity.
NOT NULL	Ensures a column cannot store a NULL (empty/unknown) value.
UNIQUE	Ensures all values in a column are different from one another (no duplicates), though unlike a primary key, a column with UNIQUE may allow one NULL.
CHECK	Ensures that all values in a column satisfy a specific Boolean condition before being accepted.
DEFAULT	Provides a default value for a column when no value is specified during insertion.

Syntax for Declaring Constraints

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE,
    Location VARCHAR(50)
);

CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    DeptID INT,
    Designation VARCHAR(30),
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    DOJ DATE,
    CONSTRAINT fk_emp_dept FOREIGN KEY (DeptID)
        REFERENCES Department(DeptID)
        ON DELETE CASCADE ON UPDATE CASCADE
);
```

Artificial Intelligence and Machine Learning | Semester III

Procedure

1. Ensure the CompanyDB database created in Practical 1 is selected using USE CompanyDB;.
2. Drop the previously created tables (in child-to-parent order, to avoid foreign key errors) using DROP TABLE, so the tables can be recreated with constraints.
3. Recreate the Department table with DeptID as PRIMARY KEY and DeptName as NOT NULL and UNIQUE.
4. Recreate the Employee table with EmpID as PRIMARY KEY, DeptID as a FOREIGN KEY referencing Department(DeptID), Salary constrained with CHECK (Salary > 0), and EmpName as NOT NULL.
5. Recreate the Product table with ProductID as PRIMARY KEY, ProductName as NOT NULL, and UnitPrice constrained with CHECK (UnitPrice > 0).
6. Recreate the Sales table with SaleID as PRIMARY KEY, ProductID and EmpID as FOREIGN KEYs referencing Product and Employee respectively, and Quantity constrained with CHECK (Quantity > 0).
7. Re-insert the sample data from Practical 1 into all four tables (parent tables first: Department and Product before Employee and Sales).
8. Deliberately attempt an INSERT that violates a constraint (for example, a duplicate primary key, a NULL in a NOT NULL column, or a foreign key value with no matching parent row) and observe the error raised by MySQL.
9. Use DESCRIBE table_name; to confirm that the Key column now shows PRI or MUL as appropriate.

Program / Commands

Step 2: Drop existing tables (child tables first)

```
DROP TABLE IF EXISTS Sales;  
DROP TABLE IF EXISTS Employee;  
DROP TABLE IF EXISTS Product;  
DROP TABLE IF EXISTS Department;
```

Step 3–6: Recreate tables with constraints

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE,  
    Location VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    EmpID INT PRIMARY KEY,  
    EmpName VARCHAR(50) NOT NULL,  
    DeptID INT,  
    Designation VARCHAR(30),  
    Salary DECIMAL(10,2) CHECK (Salary > 0),  
    DOJ DATE,  
    CONSTRAINT fk_emp_dept FOREIGN KEY (DeptID)  
        REFERENCES Department(DeptID)  
        ON DELETE CASCADE ON UPDATE CASCADE  
);  
  
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(50) NOT NULL,  
    Category VARCHAR(30),  
    UnitPrice DECIMAL(10,2) CHECK (UnitPrice > 0)  
);  
  
CREATE TABLE Sales (  
    SaleID INT PRIMARY KEY,  
    ProductID INT,  
    EmpID INT,  
    Quantity INT CHECK (Quantity > 0),  
    SaleDate DATE,  
    TotalAmount DECIMAL(10,2),  
    CONSTRAINT fk_sales_product FOREIGN KEY (ProductID)  
        REFERENCES Product(ProductID),  
    CONSTRAINT fk_sales_emp FOREIGN KEY (EmpID)  
        REFERENCES Employee(EmpID)  
);
```

Step 7: Re-insert sample data (parent tables first)

```
INSERT INTO Department (DeptID, DeptName, Location) VALUES  
(1, 'Sales', 'Mumbai'), (2, 'IT', 'Pune'), (3, 'HR', 'Delhi'),  
(4, 'Finance', 'Bengaluru'), (5, 'Marketing', 'Chennai');
```

Artificial Intelligence and Machine Learning | Semester III

```
INSERT INTO Product (ProductID, ProductName, Category, UnitPrice) VALUES  
(201, 'Laptop', 'Electronics', 55000.00), (202, 'Office Chair', 'Furniture', 4500.00),  
(203, 'Printer', 'Electronics', 12000.00), (204, 'Desk', 'Furniture', 7000.00),  
(205, 'Monitor', 'Electronics', 9000.00);
```

```
INSERT INTO Employee (EmpID, EmpName, DeptID, Designation, Salary, DOJ) VALUES  
(101, 'Aditi Sharma', 1, 'Sales Executive', 45000.00, '2021-06-14'),  
(102, 'Rohan Mehta', 2, 'Software Engineer', 65000.00, '2020-03-01'),  
(103, 'Kavya Iyer', 3, 'HR Manager', 58000.00, '2019-11-20'),  
(104, 'Sameer Khan', 4, 'Accountant', 50000.00, '2022-01-10'),  
(105, 'Neha Verma', 1, 'Sales Executive', 47000.00, '2021-08-05'),  
(106, 'Arjun Nair', 2, 'Software Engineer', 70000.00, '2018-07-23');
```

```
INSERT INTO Sales (SaleID, ProductID, EmpID, Quantity, SaleDate, TotalAmount) VALUES  
(301, 201, 101, 2, '2023-01-15', 110000.00),  
(302, 203, 105, 1, '2023-02-10', 12000.00),  
(303, 202, 101, 3, '2023-02-18', 13500.00),  
(304, 205, 105, 4, '2023-03-05', 36000.00),  
(305, 204, 101, 1, '2023-03-20', 7000.00);
```

Implementation

[Insert Figure 1 here — successful table creation with constraints and DESCRIBE output showing PRI/MUL keys]

[Insert Figure 2 here — error message produced by a primary key / NOT NULL violation]

[Insert Figure 3 here — error message produced by a foreign key / CHECK constraint violation]

Conclusion

This Practical demonstrated how PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK constraints are declared in MySQL and how they protect a database from invalid or inconsistent data. By deliberately triggering constraint violations, it was observed that MySQL rejects operations that would compromise entity integrity (duplicate/NULL primary keys), referential integrity (invalid foreign key references), or domain integrity (values outside an allowed range), confirming the practical importance of constraints in real-world database design.

Artificial Intelligence and Machine Learning | Semester III

Practical 3: Basic SQL Operations — SELECT, WHERE, ORDER BY, GROUP BY, HAVING

Title

Performing basic SQL retrieval operations on the CompanyDB database using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING clauses.

Aim

To learn how to retrieve, filter, sort, and group data stored in relational tables using the SELECT statement together with the WHERE, ORDER BY, GROUP BY, and HAVING clauses.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

The SELECT Statement

The SELECT statement is used to query and retrieve data from one or more tables. Its general syntax, showing the clauses covered in this Practical, is:

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition  
GROUP BY column(s)  
HAVING group_condition  
ORDER BY column(s) [ASC | DESC];
```

The WHERE Clause

WHERE filters individual rows before any grouping occurs, based on a condition built from comparison operators (=, <>, >, <, >=, <=), logical operators (AND, OR, NOT), and pattern/range/set operators such as BETWEEN, IN, and LIKE.

Artificial Intelligence and Machine Learning | Semester III

The ORDER BY Clause

ORDER BY sorts the final result set by one or more columns, in ascending order (ASC, the default) or descending order (DESC). Multiple columns can be specified to break ties, e.g. ORDER BY DeptID ASC, Salary DESC.

The GROUP BY Clause

GROUP BY arranges rows that share the same value in one or more specified columns into groups, typically so that an aggregate function (COUNT, SUM, AVG, MIN, MAX — studied in detail in Practical 4) can be applied to each group separately rather than to the whole table.

The HAVING Clause

HAVING filters groups after GROUP BY has formed them, typically using a condition on an aggregate value (for example, HAVING COUNT(*) > 2). It plays the same role for groups that WHERE plays for individual rows, and is required (rather than WHERE) whenever the filter condition itself involves an aggregate function.

Procedure

1. Open MySQL Workbench/CLI and select the CompanyDB database using USE CompanyDB;.
2. Write and execute a simple SELECT query to display specific columns of the Employee table.
3. Write and execute a SELECT query using WHERE to filter employees drawing a salary above a given threshold.
4. Write and execute a SELECT query using WHERE with AND/OR to combine multiple conditions.
5. Write and execute a SELECT query using ORDER BY to sort employees by salary in descending order.
6. Write and execute a SELECT query using GROUP BY to group employees department-wise.

Artificial Intelligence and Machine Learning | Semester III

7. Write and execute a SELECT query using GROUP BY together with HAVING to display only those departments having more than one employee.
8. Observe and note the result set produced by each query.

Program / Commands

(a) Basic SELECT — display specific columns

```
SELECT EmpName, Designation, Salary  
FROM Employee;
```

(b) SELECT with WHERE — simple condition

```
-- Employees earning more than 50,000  
SELECT EmpName, Salary  
FROM Employee  
WHERE Salary > 50000;
```

(c) SELECT with WHERE — compound condition (AND / OR)

```
-- Employees in Department 1 (Sales) earning more than 40,000  
SELECT EmpName, DeptID, Salary  
FROM Employee  
WHERE DeptID = 1 AND Salary > 40000;  
  
-- Employees who are Software Engineers OR HR Managers  
SELECT EmpName, Designation  
FROM Employee  
WHERE Designation = 'Software Engineer' OR Designation = 'HR Manager';
```

(d) SELECT with ORDER BY

```
-- List employees from highest to lowest salary  
SELECT EmpName, Salary  
FROM Employee  
ORDER BY Salary DESC;
```

(e) SELECT with GROUP BY

```
-- Number of employees in each department  
SELECT DeptID, COUNT(*) AS NumEmployees  
FROM Employee  
GROUP BY DeptID;
```

Artificial Intelligence and Machine Learning | Semester III

(f) SELECT with GROUP BY and HAVING

```
-- Departments having more than 1 employee  
SELECT DeptID, COUNT(*) AS NumEmployees  
FROM Employee  
GROUP BY DeptID  
HAVING COUNT(*) > 1;
```

Implementation

[Insert Figure 1 here — result grid of the WHERE-filtered query]

[Insert Figure 2 here — result grid of the ORDER BY query]

[Insert Figure 3 here — result grid of the GROUP BY ... HAVING query]

Conclusion

This Practical provided hands-on practice with the fundamental data-retrieval clauses of SQL. It was observed that WHERE filters rows before grouping while HAVING filters groups after aggregation, and that ORDER BY, applied last in logical execution, determines the final presentation order of the result set. These clauses form the backbone of virtually every SQL query used in real-world reporting and analytics.

Artificial Intelligence and Machine Learning | Semester III

Practical 4: Using Aggregate Functions

Title

Using aggregate functions — COUNT, SUM, AVG, MIN, MAX — combined with GROUP BY on the CompanyDB database.

Aim

To understand and apply SQL aggregate functions to summarize data across groups of rows, and to combine them with GROUP BY to produce department-wise and category-wise statistical summaries.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

What Are Aggregate Functions?

An aggregate function performs a calculation on a set (group) of rows and returns a single summary value. Unlike scalar functions, which operate on each row independently, aggregate functions collapse many rows into one result per group — or into a single overall result if no GROUP BY is used.

Commonly Used Aggregate Functions

Function	Purpose
COUNT(*)	Counts the number of rows in a group (including rows with NULLs in any column).
COUNT(column)	Counts the number of non-NULL values in the specified column.
SUM(column)	Returns the total of all numeric values in the column.
AVG(column)	Returns the arithmetic mean (average) of all numeric values in the column.
MIN(column)	Returns the smallest value in the column.
MAX(column)	Returns the largest value in the column.

Artificial Intelligence and Machine Learning | Semester III

5. Procedure

1. Select the CompanyDB database using USE CompanyDB;.
2. Write and execute a query using COUNT(*) to find the total number of employees.
3. Write and execute a query using SUM() to find the total sales amount recorded in the Sales table.
4. Write and execute a query using AVG() to find the average unit price of products.
5. Write and execute a query using MIN() and MAX() together to find the lowest and highest employee salaries.
6. Combine COUNT(), SUM(), and AVG() with GROUP BY to produce a department-wise summary of employee counts and average salary.
7. Combine SUM() with GROUP BY and HAVING on the Sales table to identify products whose total sales quantity exceeds a threshold.

6. Program / Commands

(a) COUNT — total number of employees

```
SELECT COUNT(*) AS TotalEmployees  
FROM Employee;
```

(b) SUM — total sales revenue

```
SELECT SUM(TotalAmount) AS TotalRevenue  
FROM Sales;
```

(c) AVG — average product price

```
SELECT AVG(UnitPrice) AS AvgProductPrice  
FROM Product;
```

(d) MIN and MAX — salary range

```
SELECT MIN(Salary) AS LowestSalary, MAX(Salary) AS HighestSalary  
FROM Employee;
```

Artificial Intelligence and Machine Learning | Semester III

(e) Aggregates with GROUP BY — department-wise summary

```
SELECT DeptID,  
       COUNT(*)    AS NumEmployees,  
       AVG(Salary) AS AvgSalary,  
       SUM(Salary) AS TotalSalaryPayout  
FROM Employee  
GROUP BY DeptID;
```

(f) Aggregates with GROUP BY and HAVING — best-selling products

```
SELECT ProductID, SUM(Quantity) AS TotalUnitsSold  
FROM Sales  
GROUP BY ProductID  
HAVING SUM(Quantity) >= 2  
ORDER BY TotalUnitsSold DESC;
```

Implementation

[Insert Figure 1 here — result of the COUNT/SUM/AVG single-value queries]

[Insert Figure 2 here — department-wise GROUP BY summary result]

[Insert Figure 3 here — GROUP BY with HAVING result for best-selling products]

Conclusion

This Practical demonstrated the use of SQL's built-in aggregate functions — COUNT, SUM, AVG, MIN, and MAX — both on an entire table and, more powerfully, on groups of rows defined by GROUP BY.

Practical 5: Performing JOIN Operations

Title

Performing INNER JOIN, LEFT JOIN, and RIGHT JOIN operations across the Employee, Department, Product, and Sales tables.

Aim

To understand the concept of combining data from multiple related tables using JOIN operations, and to practically implement INNER JOIN, LEFT JOIN, and RIGHT JOIN in MySQL.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

Why Are Joins Needed?

In a normalized relational database, related information is deliberately split across multiple tables to avoid redundancy (for example, department names are stored once in Department, not repeated in every Employee row). A JOIN operation recombines rows from two or more tables based on a related column between them typically a primary key in one table matching a foreign key in another so that meaningful, combined results can be retrieved in a single query.

Join Type	Rows Returned
INNER JOIN	Only rows with a match in both tables.
LEFT JOIN	All rows from the left table; unmatched right-side columns are NULL.
RIGHT JOIN	All rows from the right table; unmatched left-side columns are NULL.

```
SELECT columns  
FROM tableA  
INNER JOIN tableB ON tableA.key = tableB.key;
```

Artificial Intelligence and Machine Learning | Semester III

Procedure

1. Select the CompanyDB database using USE CompanyDB;.
2. Write and execute an INNER JOIN query to list each employee together with the name of their department.
3. Write and execute an INNER JOIN query across three tables (Sales, Employee, Product) to produce a readable sales report.
4. Temporarily insert one department with no employees (or observe DeptID 5, Marketing, which has none) to prepare for testing outer joins.
5. Write and execute a LEFT JOIN query from Department to Employee to list every department, including those with no employees assigned.
6. Write and execute a RIGHT JOIN query between Employee and Sales to list every employee, including those who have not made a sale, by placing Sales on the right.
7. Compare the row counts and NULL patterns produced by each join type.

Program / Commands

(a) INNER JOIN — employees with department names

```
SELECT e.EmpName, e.Designation, d.DeptName, d.Location  
FROM Employee e  
INNER JOIN Department d ON e.DeptID = d.DeptID;
```

(b) INNER JOIN across three tables — readable sales report

```
SELECT s.SaleID, p.ProductName, e.EmpName, s.Quantity, s.SaleDate, s.TotalAmount  
FROM Sales s  
INNER JOIN Product p ON s.ProductID = p.ProductID  
INNER JOIN Employee e ON s.EmpID = e.EmpID;
```

(c) LEFT JOIN — all departments, even those with no employees

```
SELECT d.DeptID, d.DeptName, e.EmpName  
FROM Department d  
LEFT JOIN Employee e ON d.DeptID = e.DeptID;  
-- Marketing (DeptID 5) has no employees, so e.EmpName will show as NULL  
-- for that row, but the Marketing row itself is still returned.
```

Artificial Intelligence and Machine Learning | Semester III

(d) RIGHT JOIN — all employees, even those with no recorded sale

```
SELECT e.EmpName, s.SaleID, s.TotalAmount
FROM Sales s
RIGHT JOIN Employee e ON s.EmpID = e.EmpID;
-- Employees such as Kavya Iyer and Sameer Khan, who have made no sale,
-- still appear, with NULL in the SaleID and TotalAmount columns.
```

Implementation

[Insert Figure 1 here — INNER JOIN result combining Employee and Department]

[Insert Figure 2 here — LEFT JOIN result showing the unmatched Marketing department with NULLs]

[Insert Figure 3 here — RIGHT JOIN result showing employees with no matching sales]

Conclusion

This Practical demonstrated how INNER JOIN, LEFT JOIN, and RIGHT JOIN combine related data spread across multiple normalized tables.

Practical 6: Working with Subqueries and Nested Queries

Title

Writing single-row, multiple-row, and correlated subqueries on the CompanyDB database.

Aim

To understand what a subquery (nested query) is, and to write single-row, multi-row, and correlated subqueries to solve problems that cannot easily be expressed with a single flat SELECT statement.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

What Is a Subquery?

A subquery (also called a nested query or inner query) is a SELECT statement embedded inside another SQL statement typically within the WHERE, HAVING, FROM, or SELECT clause of an outer query.

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

Multiple-Row Subquery

```
SELECT ProductName
FROM Product
WHERE ProductID IN (SELECT ProductID FROM Sales WHERE Quantity >= 3);
```

Correlated Subquery

```
SELECT e.EmpName
FROM Employee e
WHERE EXISTS (
    SELECT 1 FROM Sales s WHERE s.EmpID = e.EmpID
);
```

Artificial Intelligence and Machine Learning | Semester III

Procedure

1. Select the CompanyDB database using USE CompanyDB;.
2. Write and execute a single-row subquery to list employees earning more than the average salary.
3. Write and execute a single-row subquery using MAX() to identify the highest-paid employee.
4. Write and execute a multi-row subquery using IN to list products that have appeared in at least one sale of quantity 3 or more.
5. Write and execute a multi-row subquery using NOT IN to list employees who have never made a sale.
6. Write and execute a correlated subquery using EXISTS to list employees who have made at least one sale.
7. Compare the result of the EXISTS-based correlated subquery with an equivalent INNER JOIN, and the NOT IN subquery with an equivalent LEFT JOIN ... WHERE ... IS NULL.

Program / Commands

(a) Single-row subquery — above-average earners

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

(b) Single-row subquery — highest-paid employee

```
SELECT EmpName, Salary
FROM Employee
WHERE Salary = (SELECT MAX(Salary) FROM Employee);
```

(c) Multi-row subquery — products sold in bulk (IN)

```
SELECT ProductName
FROM Product
WHERE ProductID IN (
    SELECT ProductID FROM Sales WHERE Quantity >= 3
);
```

Artificial Intelligence and Machine Learning | Semester III

(d) Multi-row subquery — employees with no sales (NOT IN)

```
SELECT EmpName
FROM Employee
WHERE EmpID NOT IN (
    SELECT DISTINCT EmpID FROM Sales
);
```

(e) Correlated subquery — employees who have made at least one sale (EXISTS)

```
SELECT e.EmpName
FROM Employee e
WHERE EXISTS (
    SELECT 1 FROM Sales s WHERE s.EmpID = e.EmpID
);
```

Implementation

[Insert Figure 1 here — result of the above-average-salary single-row subquery]

[Insert Figure 2 here — result of the IN-based multi-row subquery on products]

[Insert Figure 3 here — result of the EXISTS-based correlated subquery]

Conclusion

This Practical demonstrated how subqueries allow complex, multi-step questions to be answered within a single SQL statement, using single-row subqueries for scalar comparisons, multi-row subqueries with IN/NOT IN for set membership, and correlated subqueries with EXISTS for row-by-row dependent evaluation.

Practical 7: ER Diagram Design and Mapping to Relational Tables

Title

Designing an Entity-Relationship (ER) diagram for a Library Management System and mapping it to relational (table) schema.

Aim

To design an Entity-Relationship (ER) diagram for a sample application — a Library Management System — identifying its entities, attributes, and relationships, and to translate that conceptual design into a set of relational tables using standard ER-to-relational mapping rules.

Software / Tools Used

- A diagramming tool such as draw.io, MySQL Workbench (EER Diagram feature), or pen and paper
- MySQL Server 8.0 or later (to implement the mapped schema)

Theory

What Is an ER Diagram?

An Entity-Relationship (ER) diagram is a conceptual, graphical representation of the data requirements of a system, developed before any physical database is built. It identifies the real-world objects (entities) the system needs to store information about, the properties of those objects (attributes), and the associations between them (relationships), independent of how the data will eventually be implemented in tables.

4.2 Basic ER Notation

Symbol	Meaning
Rectangle	Represents an entity — a real-world object or concept about which data is stored (e.g., BOOK, MEMBER).
Oval / list under entity	Represents an attribute — a property of an entity (e.g., Title, Author). An underlined attribute is the primary key.
Diamond	Represents a relationship — an association between two or more entities (e.g., ISSUES, BELONGS TO).

Artificial Intelligence and Machine Learning | Semester III

Symbol	Meaning
Line with cardinality label (1, N, M)	Connects an entity to a relationship and indicates how many instances of one entity can be associated with instances of the other (1:1, 1:N, or M:N).

5. Procedure

1. Identify the entities relevant to the chosen application (Library Management System): BOOK, CATEGORY, MEMBER, and STAFF.
2. Identify the key and non-key attributes of each entity.
3. Identify the relationships between entities and determine the cardinality of each: BOOK BELONGS TO CATEGORY (N:1), STAFF REGISTERS MEMBER (1:N), and MEMBER ISSUES BOOK (M:N).
4. Draw the ER diagram using the notation described above, showing entities as rectangles, relationships as diamonds, attributes listed inside each entity box, and cardinality labels on the connecting lines.
5. Apply the ER-to-relational mapping rules to derive the relational schema: convert each entity to a table, resolve each 1:N relationship with a foreign key, and resolve the M:N relationship with a junction table.
6. Implement the derived relational schema in MySQL using CREATE TABLE statements.

6. Program / Commands

ER Diagram

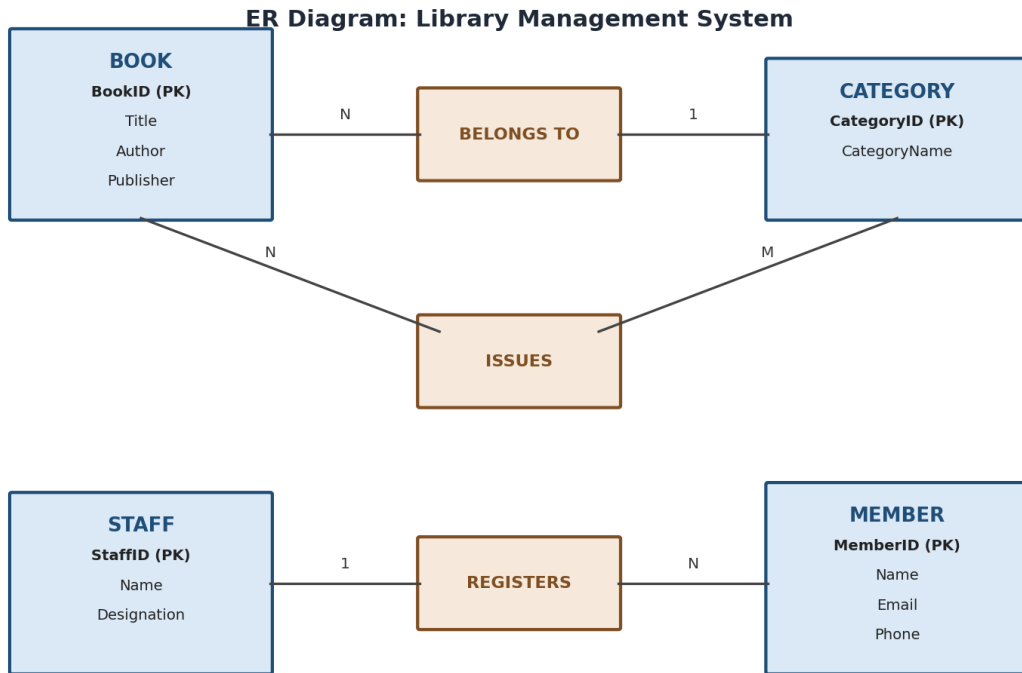


Figure 4: ER Diagram of the Library Management System

DDL for the mapped schema

```

CREATE DATABASE LibraryDB;
USE LibraryDB;

CREATE TABLE Category (
    CategoryID INT PRIMARY KEY,
    CategoryName VARCHAR(40) NOT NULL
);

CREATE TABLE Staff (
    StaffID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Designation VARCHAR(30)
);

CREATE TABLE Book (

```

Artificial Intelligence and Machine Learning | Semester III

```
BookID    INT PRIMARY KEY,
Title     VARCHAR(100) NOT NULL,
Author    VARCHAR(60),
Publisher  VARCHAR(60),
CategoryID INT,
CONSTRAINT fk_book_category FOREIGN KEY (CategoryID)
REFERENCES Category(CategoryID)
);

CREATE TABLE Member (
  MemberID    INT PRIMARY KEY,
  Name        VARCHAR(50) NOT NULL,
  Email       VARCHAR(60) UNIQUE,
  Phone       VARCHAR(15),
  RegisteredBy INT,
  CONSTRAINT fk_member_staff FOREIGN KEY (RegisteredBy)
REFERENCES Staff(StaffID)
);

-- Junction table resolving the M:N "ISSUES" relationship
CREATE TABLE IssueRecord (
  BookID    INT,
  MemberID  INT,
  IssueDate DATE NOT NULL,
  DueDate   DATE,
  ReturnDate DATE,
  PRIMARY KEY (BookID, MemberID, IssueDate),
  CONSTRAINT fk_issue_book FOREIGN KEY (BookID) REFERENCES Book(BookID),
  CONSTRAINT fk_issue_member FOREIGN KEY (MemberID) REFERENCES
Member(MemberID)
);
```

Implementation

The ER diagram itself is provided above as Figure 4. Insert the following additional screenshots to document the implementation:

[Insert Figure 5 here — successful execution of the mapped-schema DDL script in MySQL Workbench]

[Insert Figure 6 here — DESCRIBE output for the IssueRecord junction table showing the composite key]

Artificial Intelligence and Machine Learning | Semester III

Conclusion

This Practical demonstrated the complete conceptual-to-logical database design workflow: identifying entities, attributes, and relationships for a real-world application; representing them in a standard ER diagram; and systematically translating that diagram into a normalized relational schema using well-defined mapping rules — including the important special case of resolving a many-to-many relationship through a junction table. This workflow is the standard first step in professional database design before any SQL is written.

Artificial Intelligence and Machine Learning | Semester III

Practical 8: Normalization — 1NF, 2NF, 3NF, and BCNF

Title

Normalizing an unnormalized Student-Course-Instructor relation through the First, Second, and Third Normal Forms, and studying Boyce-Codd Normal Form (BCNF) through a classic case study.

Aim

To understand the concept of functional dependency and data redundancy anomalies, and to apply the rules of First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and Boyce-Codd Normal Form (BCNF) to progressively decompose an unnormalized relation into a well-structured set of relations.

Software / Tools Used

- Pen-and-paper / whiteboard design (normalization is primarily a conceptual, design-time exercise)
- MySQL Server 8.0 or later (to implement the final normalized schema)

Theory

Why Normalize? — Update Anomalies

An unnormalized or under-normalized table that mixes multiple, unrelated facts together suffers from three classic anomalies:

- Insertion anomaly: it may be impossible to record a fact (e.g., add a new course with no students yet enrolled) without also recording an unrelated fact.
- Update anomaly: because the same fact is repeated across many rows (e.g., an instructor's room number repeated once per enrolled student), updating it requires changing many rows consistently, risking inconsistency if even one is missed.
- Deletion anomaly: deleting one fact (e.g., the last student enrolled in a course) may unintentionally erase another, unrelated fact (e.g., the only record of that course's existence).

Artificial Intelligence and Machine Learning | Semester III

Normalization is the systematic process of decomposing relations to eliminate these anomalies, by ensuring that each relation stores facts about only one type of thing.

Normal Form	Requirement
1NF	All attribute values are atomic; no repeating groups.
2NF	1NF + no partial dependency of a non-key attribute on part of a composite key.
3NF	2NF + no transitive dependency of a non-key attribute on another non-key attribute.
BCNF	Every determinant (left-hand side of any functional dependency) is a candidate key.

Procedure

1. Start with an unnormalized StudentEnrollment relation in which each student row stores a repeating group of enrolled courses.
2. Convert the relation to 1NF by flattening the repeating group so that each (Student, Course) pair occupies its own row.
3. Identify the functional dependencies present and the resulting partial dependencies, then decompose the 1NF relation into 2NF by separating out attributes that depend on only part of the composite key.
4. Identify any transitive dependency remaining in the 2NF relations and decompose further into 3NF by separating out the transitively dependent attribute into its own relation.
5. Verify that each 3NF relation also satisfies BCNF by checking whether every determinant is a candidate key; where it is not (as in the Student-Subject-Teacher case study), decompose further into BCNF.
6. Implement the final, fully normalized set of relations in MySQL using CREATE TABLE statements with appropriate primary and foreign keys.

Artificial Intelligence and Machine Learning | Semester III

Program / Commands

Step 1: Unnormalized relation (0NF)

Consider the following unnormalized StudentEnrollment table, where the Courses column stores a repeating group — multiple course codes packed into a single cell for one student:

StudentID	StudentName	Courses (CourseID : CourseName : Instructor : Room)
S1	Aditi Sharma	C1:DBMS:Prof. Rao:R101, C2:OS:Prof. Iyer:R102
S2	Rohan Mehta	C1:DBMS:Prof. Rao:R101

Step 2: 1NF — flatten repeating groups

Each (Student, Course) combination is given its own row, with every column holding a single atomic value. The composite primary key is (StudentID, CourseID).

StudentID	StudentName	CourseID	CourseName	Instructor	Room	Grade
S1	Aditi Sharma	C1	DBMS	Prof. Rao	R101	A
S1	Aditi Sharma	C2	OS	Prof. Iyer	R102	B
S2	Rohan Mehta	C1	DBMS	Prof. Rao	R101	A

Functional dependencies present: StudentID → StudentName (partial — depends only on part of the key); CourseID → CourseName, Instructor, Room (partial); (StudentID, CourseID) → Grade (full dependency on the whole key). Because StudentName and the course details depend on only part of the composite key, this relation is in 1NF but not 2NF.

Step 3: 2NF — remove partial dependencies

The relation is decomposed so that every non-key attribute depends on the whole of its table's key:

```
Student(StudentID, StudentName)
Course(CourseID, CourseName, Instructor, Room)
Enrollment(StudentID, CourseID, Grade)
```

Step 4: 3NF — remove transitive dependencies

The transitively dependent attribute is moved into its own relation, keyed on the attribute it truly depends on:

```
Student(StudentID, StudentName)
```

Artificial Intelligence and Machine Learning | Semester III

```
Enrollment(StudentID, CourseID, Grade)
Course(CourseID, CourseName, Instructor)
InstructorRoom(Instructor, Room)
```

All four relations are now in 3NF: every non-key attribute depends directly on its own table's primary key, with no partial or transitive dependency remaining.

Step 5: BCNF case study — when 3NF is not enough

Consider a separate, classic case that shows 3NF and BCNF can differ. Let StudentSubjectTeacher(Student, Subject, Teacher) record which teacher teaches which subject to which student, with two functional dependencies: (Student, Subject) → Teacher, and Teacher → Subject (a business rule stating each teacher teaches only one subject). This relation has two candidate keys, {Student, Subject} and {Student, Teacher}.

BCNF decomposition (splitting out the dependency whose determinant is not a superkey):

```
TeacherSubject(Teacher, Subject)    -- Teacher -> Subject
StudentTeacher(Student, Teacher)    -- records which teacher a student is taught by
```

Step 6: Implementing the final normalized schema

```
CREATE DATABASE NormalizationDemo;
USE NormalizationDemo;

CREATE TABLE Student (
    StudentID VARCHAR(10) PRIMARY KEY,
    StudentName VARCHAR(50) NOT NULL
);

CREATE TABLE InstructorRoom (
    Instructor VARCHAR(50) PRIMARY KEY,
    Room VARCHAR(10)
);

CREATE TABLE Course (
    CourseID VARCHAR(10) PRIMARY KEY,
    CourseName VARCHAR(50) NOT NULL,
    Instructor VARCHAR(50),
    CONSTRAINT fk_course_instructor FOREIGN KEY (Instructor)
        REFERENCES InstructorRoom(Instructor)
);
```

Artificial Intelligence and Machine Learning | Semester III

```
CREATE TABLE Enrollment (  
    StudentID VARCHAR(10),  
    CourseID VARCHAR(10),  
    Grade CHAR(2),  
    PRIMARY KEY (StudentID, CourseID),  
    CONSTRAINT fk_enr_student FOREIGN KEY (StudentID) REFERENCES  
Student(StudentID),  
    CONSTRAINT fk_enr_course FOREIGN KEY (CourseID) REFERENCES  
Course(CourseID)  
);
```

Implementation

[Insert Figure 7 here — hand-drawn or diagrammed decomposition from the unnormalized table through 1NF, 2NF, and 3NF]

[Insert Figure 8 here — successful execution of the final normalized CREATE TABLE script in MySQL]

Conclusion

This Practical traced a single unnormalized relation through 1NF, 2NF, and 3NF, removing repeating groups, partial dependencies, and transitive dependencies at each stage, and then used a classic Student-Subject-Teacher case study to show why BCNF is a strictly stronger condition than 3NF.

Artificial Intelligence and Machine Learning | Semester III

Practical 9: Managing Transactions Using COMMIT, ROLLBACK, and SAVEPOINT

Title

Managing transactions in SQL using the Transaction Control Language (TCL) commands COMMIT, ROLLBACK, and SAVEPOINT.

Aim

To understand the concept of a database transaction and the ACID properties, and to practically use the TCL commands COMMIT, ROLLBACK, and SAVEPOINT to control when changes made to the database are made permanent or undone.

Software / Tools Used

- MySQL Server 8.0 or later
- MySQL Workbench 8.0 or MySQL Command Line Client

Theory

What Is a Transaction?

A transaction is a sequence of one or more SQL operations (typically INSERT, UPDATE, and/or DELETE statements) that are executed as a single, indivisible logical unit of work.

The ACID Properties

Property	Meaning
Atomicity	A transaction is treated as a single, indivisible unit — it either completes entirely or has no effect at all.
Consistency	A transaction takes the database from one valid state to another valid state, preserving all defined rules and constraints.
Isolation	Concurrently executing transactions do not interfere with each other's intermediate, uncommitted results.
Durability	Once a transaction is committed, its changes are permanently saved and will survive any subsequent system failure.

Artificial Intelligence and Machine Learning | Semester III

Autocommit Mode

By default, MySQL runs in autocommit mode, where every individual SQL statement is automatically treated as its own transaction and committed immediately upon successful execution. To group multiple statements into a single, manually controlled transaction, autocommit must be turned off (SET autocommit = 0;) or a transaction must be explicitly started with START TRANSACTION; (equivalently, BEGIN;).

COMMIT

The COMMIT statement permanently saves all changes made by the current transaction to the database. Once committed, the changes cannot be undone by ROLLBACK — they become a durable part of the database.

ROLLBACK

The ROLLBACK statement undoes all changes made by the current transaction since it began (or since the last commit), restoring the database to the state it was in before the transaction started. This is essential for recovering from errors detected partway through a multi-step operation.

SAVEPOINT

A SAVEPOINT marks an intermediate point within a transaction that can later be rolled back to, without discarding the entire transaction. The statement ROLLBACK TO SAVEPOINT savepoint_name; undoes only the changes made after that savepoint was set, while changes made before it remain intact and the transaction remains open, allowing further statements (and an eventual COMMIT) to follow.

Procedure

1. Select the CompanyDB database using USE CompanyDB;.
2. Turn off autocommit (or issue START TRANSACTION;) so that a sequence of statements can be treated as a single transaction.
3. Insert a new department row as part of the transaction and query the table to confirm the change is visible within the same session.

Artificial Intelligence and Machine Learning | Semester III

4. Issue COMMIT; and verify (in a new session, if possible) that the new department row now permanently exists.
5. Start a second transaction; update an employee's salary, then set a SAVEPOINT.
6. After the savepoint, delete a row from the Sales table.
7. Issue ROLLBACK TO SAVEPOINT to undo only the deletion, while keeping the earlier salary update pending within the transaction.
8. Query the tables to confirm that the deleted Sales row has reappeared, while the salary update is still present (but not yet permanent).
9. Issue a final ROLLBACK; to undo the entire remaining transaction, and verify that the salary update itself is also discarded, restoring the original value.

Program / Commands

(a) COMMIT — making a change permanent

```
USE CompanyDB;
START TRANSACTION;

INSERT INTO Department (DeptID, DeptName, Location)
VALUES (6, 'Logistics', 'Hyderabad');

SELECT * FROM Department; -- new row is visible within this session

COMMIT; -- change is now permanent and durable

SELECT * FROM Department; -- row 6 persists even in a new session
```

(b) ROLLBACK — undoing an entire transaction

```
START TRANSACTION;

DELETE FROM Sales WHERE SaleID = 305;
SELECT * FROM Sales;      -- SaleID 305 appears to be gone

ROLLBACK;                -- undo the DELETE entirely

SELECT * FROM Sales;      -- SaleID 305 is restored
```

Artificial Intelligence and Machine Learning | Semester III

(c) SAVEPOINT — partial rollback within a transaction

```
START TRANSACTION;

-- Step 1: update a salary
UPDATE Employee SET Salary = Salary + 5000 WHERE EmpID = 101;

-- Mark a savepoint after the salary update
SAVEPOINT before_delete;

-- Step 2: delete a sales record
DELETE FROM Sales WHERE SaleID = 302;

SELECT * FROM Sales;      -- SaleID 302 appears removed
SELECT * FROM Employee WHERE EmpID = 101; -- salary shows the raise

-- Undo ONLY the delete, keeping the salary update pending
ROLLBACK TO SAVEPOINT before_delete;

SELECT * FROM Sales;      -- SaleID 302 has reappeared
SELECT * FROM Employee WHERE EmpID = 101; -- salary raise is still pending

-- Finally, discard the whole transaction (including the pending raise)
ROLLBACK;

SELECT * FROM Employee WHERE EmpID = 101; -- salary is back to its original value
```

Implementation

[Insert Figure 9 here — transaction and COMMIT sequence with before/after SELECT results]

[Insert Figure 10 here — ROLLBACK sequence showing a deleted row being restored]

[Insert Figure 11 here — SAVEPOINT sequence showing partial rollback, followed by full rollback]

Conclusion

This Practical demonstrated how the TCL commands COMMIT, ROLLBACK, and SAVEPOINT give a database programmer precise control over when changes become permanent and how far back an error can be undone.